

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

10. Chuyển đổi Thực tế — Từ Monolith đến Microservices	4
10.1. Thời điểm thích hợp để chuyển đổi	6
10.1.1. Vấn đề: microservices không phải đích đến mặc định	6
10.1.2. Ma trận quyết định — Khi nào cân nhắc chuyển đổi	6
10.1.3. KBM: Kiến trúc tiến hóa (Evolutionary Architecture), không phải Big Bang	6
10.1.4. Bài học thực tế — Khi chuyển đổi đi sai hướng (Anti-patterns)	7
10.2. Strangler Fig Pattern — Chuyển đổi tăng dần	8
10.2.1. Vấn đề: Chuyển đổi tức thì (“Big Bang”) so với chuyển đổi tăng dần (Incremental migration)	8
10.2.2. Strangler Fig Pattern	8
10.2.3. API Gateway — điểm tiếp nối (seam) cho Strangler Fig	9
10.2.4. KBM: Strangler Fig thực tế — từ <code>kbm-legacy-monolith</code> đến <code>kbm-*</code>	10
10.3. Tách Database — Thách thức Lớn nhất	11
10.3.1. Vấn đề: database coupling nguy hiểm hơn code coupling	11
10.3.2. Cạm bẫy ghi kép (dual-write) và Outbox Pattern	12
10.3.3. CDC cho chuyển đổi dữ liệu	14
10.4. Anti-Corruption Layer và các pattern chuyển đổi	14
10.4.1. Anti-Corruption Layer (ACL) — Bảo vệ ranh giới	14
10.4.2. Branch by Abstraction	15
10.4.3. Parallel Run — Kiểm chứng trước khi chuyển đổi	16
10.5. Quá trình chuyển đổi kiến trúc tại KBM	17
10.5.1. Tổng hợp các phân tích khoảng trống	17
10.5.2. Lộ trình chuyển đổi — 4 giai đoạn	18
10.5.3. Ma trận quyết định — Ưu tiên theo tác động × nỗ lực	20
10.6. Sai lầm thường gặp khi chuyển đổi	21
10.7. Tổng kết	21
10.8. Đọc thêm	22
Tài liệu tham khảo	23

10.

CHƯƠNG 10.

Chuyển đổi Thực tế – Từ Monolith đến Microservices

“If you can’t build a monolith, what makes you think microservices are the answer?”

— Simon Brown (trích dẫn bởi Sam Newman, [4b])

 Lưu ý

Chuyển đổi (Migration) là một *kỹ năng riêng*: áp dụng các pattern đã học ở Phần I–II vào một hệ thống *đang vận hành thực tế (production)*, nơi không thể viết lại từ đầu. Chương này thêm yếu tố **phán đoán** — khi nào nên và khi nào **KHÔNG** nên chuyển sang microservices — và là nơi trình bày **lộ trình chuyển đổi (migration roadmap) chi tiết cho KBM** (Quick Wins → Observability → Resilience → Database Decomposition). Việc tổng kết 12 chương sẽ được trình bày ở phần Kết luận. Monolith First không phủ nhận những gì đã học — mà là nguyên tắc lựa chọn thời điểm và cách áp dụng đúng ngữ cảnh.

Mục tiêu xuyên suốt: chuyển đổi nền tảng LMS mà không làm gián đoạn việc học của sinh viên. Cụ thể, chương này giúp người đọc:

MỤC TIÊU HỌC TẬP

- Hiểu khi nào nên và **khi nào KHÔNG nên** chuyển sang microservices
- Nắm vững **Strangler Fig Pattern** — chiến lược chuyển đổi tăng dần phổ biến nhất

- Áp dụng **5 chiến lược tách database** từ góc nhìn chuyển đổi thực tế
- Hiểu **Anti-Corruption Layer** và các pattern chuyển đổi bảo vệ ranh giới hệ thống
- Thiết kế **lộ trình chuyển đổi** khả thi: phân chia các giai đoạn (phases), ưu tiên theo mức độ tác động và nỗ lực (Impact × Effort)
- Phân tích lộ trình chuyển đổi cho KBM — tổng hợp các phân tích khoảng trống (gap analyses) xuyên suốt 12 chương

10.1. Thời điểm thích hợp để chuyển đổi

Quyết định nâng cấp một hệ thống nguyên khối lên kiến trúc microservices giống như việc quyết định đập bỏ một tòa nhà cũ để xây khu học xá mới. Cần phải có lý do chính đáng và thời điểm phù hợp.

10.1.1. Vấn đề: microservices không phải đích đến mặc định

Sau 9 chương phân tích các mẫu kiến trúc (patterns), cơ chế giao tiếp (communication), quản lý dữ liệu (data management), API Gateway, và bảo mật (security), độc giả có thể có ấn tượng rằng microservices là kiến trúc mọi hệ thống nên hướng tới. **Đây là một sai lầm phổ biến.**

Martin Fowler — người đồng tác giả bài viết gốc “Microservices” (2014) — cảnh báo rõ ràng [W2]:

Monolith First

“Almost all the cases where I’ve heard of a system that was built as a microservice system from scratch, it has ended up in serious trouble... You shouldn’t start a new project with microservices, even if you’re sure your application will be big enough to make it worthwhile.”

— Martin Fowler [W2]

Newman trong [4b, Ch.1] bổ sung: microservices là **một lựa chọn kiến trúc** không phải bước tiến hóa bắt buộc. Monolith không phải “kiến trúc sai” — monolith là kiến trúc đúng cho phần lớn hệ thống ở giai đoạn đầu.

10.1.2. Ma trận quyết định — Khi nào cân nhắc chuyển đổi

Tiêu chí	Monolith đủ tốt	Cân nhắc Microservices
Quy mô đội ngũ (Team size)	≤ 8 người	> 15 người, nhiều nhóm
Tần suất triển khai (Deploy frequency)	Hàng tuần/tháng	Triển khai hàng ngày theo nhóm
Yêu cầu mở rộng (Scale requirements)	Mở rộng toàn bộ là đủ	Mở rộng từng thành phần độc lập
Đa dạng công nghệ (Technology diversity)	Một bộ công nghệ (tech stack) là đủ	Cần đa ngôn ngữ (polyglot: Java + Python + Go)
Độ phức tạp nghiệp vụ (Domain complexity)	Ranh giới chưa rõ	Bounded contexts đã xác định rõ (Ch.2)
Mức độ trưởng thành của tổ chức (Organizational maturity)	Chưa có DevOps, CI/CD	Đã có CI/CD, container hóa, giám sát

Bảng 10.1: Ma trận quyết định (Decision Matrix) — Khi nào cân nhắc chuyển đổi

Richardson trong [2a, Ch.13] gọi đây là **Microservice Premium** — chi phí phải trả khi chuyển sang microservices: độ phức tạp của hệ phân tán, lỗi mạng, eventual consistency, chi phí vận hành. Khoản phí này (premium) chỉ đáng trả khi lợi ích (như triển khai độc lập, mở rộng linh hoạt, sự tự chủ của đội ngũ) vượt qua chi phí.

10.1.3. KBM: Kiến trúc tiến hóa (Evolutionary Architecture), không phải Big Bang

KBM không bắt đầu từ một bản thiết kế microservices hoàn chỉnh định sẵn. Hệ thống đã đi qua nhiều thế hệ: các judge monolith riêng cho bài lập trình, Network Judge, SQL Judge mới, rồi dần hình thành các service cho auth, assignment, judge, notification và DevOps Lab. Vì vậy câu hỏi đúng không phải là “có

nên microservices-first không?”, mà là: *mỗi bước tiến hóa có đang giải quyết đúng vấn đề của thời điểm đó không?*

Microservice Premium trong KBM

KBM chọn hướng microservices vì hai lý do cụ thể: (1) mục đích **giáo dục** — bản thân hệ thống là tài liệu học tập cho sinh viên, (2) miền nghiệp vụ (domain) cốt lõi đã đủ rõ sau nhiều năm vận hành các hệ thống chấm điểm (judge systems): Identity, SQL Practice, Network Practice, Academic Management, Evaluation/Infrastructure.

Tuy nhiên, sự đánh đổi (trade-off) hiện rõ: (1) cơ sở dữ liệu dùng chung (shared database) giữa Core Service và Assignment Service — sự phụ thuộc (coupling) vẫn tồn tại dù dịch vụ đã được tách, (2) thư viện dùng chung (shared library) chứa quá nhiều logic — rủi ro nguyên khối phân tán (distributed monolith), (3) môi trường đa ngôn ngữ (polyglot) Java+Go làm tăng chi phí biên dịch, kiểm thử và triển khai. Đây là ví dụ thực tế của Microservice Premium: **lợi ích giáo dục và nhu cầu cô lập hệ thống chấm điểm đủ để chứng minh tính hợp lý của chi phí**, nhưng vẫn cần một lộ trình giảm thiểu sự phụ thuộc (coupling) có kiểm soát.

Một bài học cụ thể nằm ở SQL Judge, không phải Network Practice. Network Practice được thiết kế độc lập để đánh giá giao tiếp qua mạng; nó minh họa tốt bounded context theo protocol. Với SQL Judge, bài toán là ranh giới giữa một bộ điều phối (coordinator) `judge` và nhiều tiến trình xử lý (workers) `judge-*` theo từng hệ quản trị cơ sở dữ liệu (DBMS) (`judge-mysql`, `judge-sqlserver`, ...). Chuyển đổi đúng hướng là làm rõ cơ chế định tuyến theo quyền sở hữu (ownership routing), chuẩn hóa giao ước (contract) của công việc và kết quả, bổ sung idempotency dựa trên `judgeRunId`, sau đó mới tiến hành mở rộng theo chiều ngang (scale) cho từng worker dựa trên khối lượng công việc (workload) của hệ quản trị cơ sở dữ liệu.

Monolith First — tổng hợp

“Start with a monolith. Move to microservices only when the monolith becomes a problem — and you know which problems you’re solving. The worst microservices systems are those built by teams who started with microservices *before* understanding their domain boundaries.”

— Tổng hợp từ Martin Fowler [W2] và Sam Newman [4b, Ch.1]

10.1.4. Bài học thực tế — Khi chuyển đổi đi sai hướng (Anti-patterns)

Dù quyết định chuyển đổi là đúng đắn, *cách thức* chuyển đổi vẫn có thể gây thất bại cho dự án. Richardson trong [2b, Ch.2] phân tích bốn anti-patterns phổ biến nhất khi chuyển từ monolith sang microservices — chúng ta minh họa qua các sai lầm *tiềm năng* nếu chia tách sai trong KBM:

Anti-pattern	Ví dụ sai lầm tiềm năng trong KBM	Hậu quả
Data Services (<i>Dịch vụ bao bọc CRUD</i>)	Tách <code>SqlExecutorService</code> thành <code>QuestionDataService</code> riêng — Core Service phải thực hiện lệnh gọi HTTP mỗi lần cần truy xuất dữ liệu câu hỏi	Tăng độ trễ (latency) mạng, mất lợi ích giao dịch (transaction) nội bộ
Fine-grained Services (<i>Chia quá nhỏ</i>)	Chia Auth thành 3 service: <code>UserProfile</code> , <code>UserCredential</code> , <code>UserSession</code>	Mỗi lần đăng nhập phải điều phối (orchestrate) 3 dịch vụ, khó gỡ lỗi (debug)

Anti-pattern	Ví dụ sai lầm tiềm năng trong KBM	Hậu quả
Microservices-first	Tách KBM theo tầng kỹ thuật (technical layer) thay vì theo miền nghiệp vụ — mỗi thay đổi chấm điểm ảnh hưởng nhiều services cùng lúc	Phát sinh “Distributed Monolith” (xem thêm §10.6)
End-to-end QA Gate	Triển khai độc lập nhưng đội kiểm thử (QA) phải chờ tích hợp tất cả để kiểm thử	Triệt tiêu khả năng triển khai độc lập (<i>Independent Deployability</i>)

Bảng 10.2: Anti-patterns khi chuyển đổi Microservices (minh họa với KBM)

Các sai lầm chuyển đổi khác (Big Bang Rewrite, Lift-and-Shift, Over-engineering) sẽ được phân tích chi tiết tại §10.6.

10.2. Strangler Fig Pattern — Chuyển đổi tăng dần

Thay vì một bước đập bỏ toàn bộ hệ thống cũ đầy rủi ro, chiến lược chuyển đổi dần dần giúp nhà trường vẫn duy trì được các hoạt động đào tạo bình thường trong lúc xây dựng nền tảng mới.

10.2.1. Vấn đề: Chuyển đổi tức thì (“Big Bang”) so với chuyển đổi tăng dần (Incremental migration)

Có hai cách thức chính để dịch chuyển từ monolith sang microservices: **Big Bang** (viết lại toàn bộ hệ thống mới và chuyển đổi cùng lúc) mang rủi ro rất cao và thường dẫn đến thất bại; hoặc **Tăng dần (Strangler Fig)** (tách dần từng phần, cho chạy song song) với rủi ro thấp hơn nhiều do có thể dễ dàng hoàn tác (rollback) từng phần.

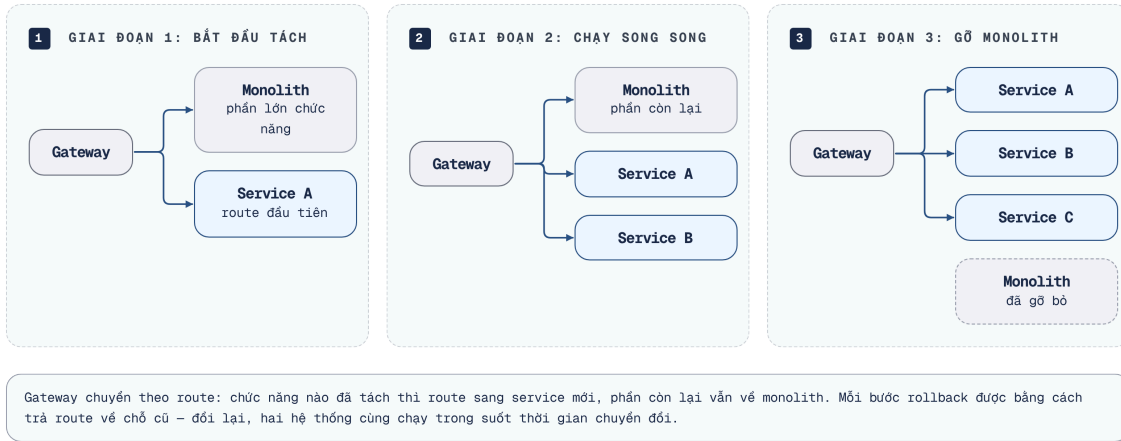
Newman trong [4b, Ch.3] và Richardson trong [2a, Ch.13] đều khẳng định: **Viết lại kiểu Big Bang gần như luôn thất bại** — “the second system effect” (Fred Brooks). Khi viết lại từ con số không, đội ngũ phát triển mất toàn bộ kiến thức nghiệp vụ (domain knowledge) được nhúng trong mã nguồn cũ, thời hạn (deadline) liên tục trễ, và khách hàng phải chờ đợi *tất cả* tính năng trước khi nhận *bất kỳ* giá trị nào.

10.2.2. Strangler Fig Pattern

📍 Xây Thư viện số hiện đại

Thay vì giải thích bằng thực vật học (cây đa búp nghệt), hãy tưởng tượng trường đại học muốn thay hệ thống Thư viện vật lý cũ bằng Thư viện số. Tòa nhà cũ không bị đập bỏ ngay lập tức (Big Bang); thay vào đó, Thư viện số được xây ngay bên cạnh (Strangler Fig). Ban đầu, khi sinh viên đến cổng trường (API Gateway) để xin “Mượn sách Điện tử”, bảo vệ chỉ họ sang tòa nhà mới. Nếu họ xin “Mượn sách Giấy”, bảo vệ vẫn chỉ vào tòa nhà cũ. Từng khu vực (Tạp chí, Giáo trình...) lần lượt được số hóa và sinh viên được chuyển hướng sang tòa nhà mới, cho đến khi tòa nhà cũ không còn ai vào nữa và bị phá dỡ hoàn toàn.

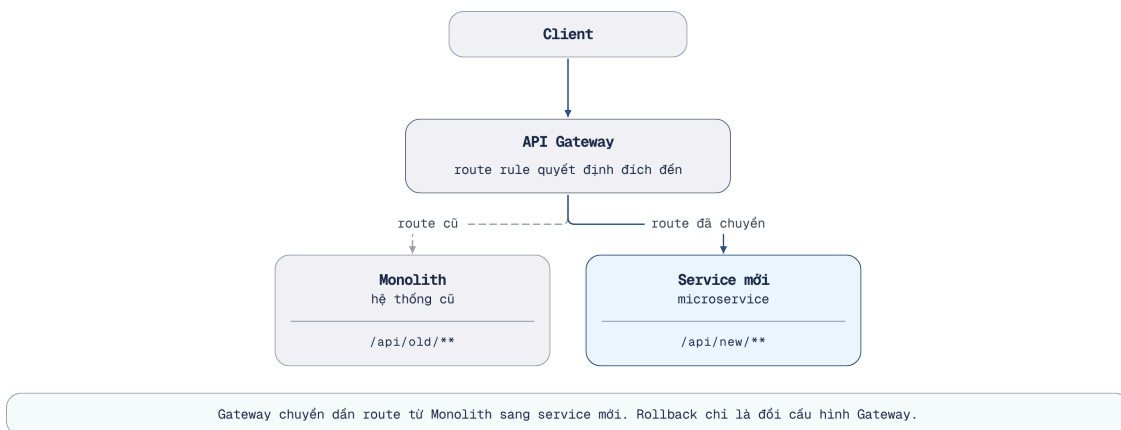
Strangler Fig — pattern do Martin Fowler đặt tên (2004), lấy cảm hứng từ cây đa búp nghệt (strangler fig) bao quanh cây chủ, dần thay thế cho đến khi cây chủ biến mất [W8].



Hình 10.1: Strangler Fig Pattern — dần thay thế monolith qua 3 giai đoạn

10.2.3. API Gateway — điểm tiếp nối (seam) cho Strangler Fig

Để áp dụng mô hình này, chúng ta cần khả năng định tuyến giao tiếp tốt. API Gateway (đã học ở Chương 8) hoạt động như một điểm tiếp nối (seam) cho Strangler Fig, giúp che giấu sự phức tạp của quá trình chuyển đổi khỏi người dùng cuối:



Hình 10.2: API Gateway làm điểm tiếp nối cho Strangler Fig — client không biết route nào đến monolith, route nào đến service mới

Tùy thuộc vào kiến trúc hiện tại, chúng ta có ba chiến lược triển khai Strangler Fig chính: **Route-based** (dùng Gateway định tuyến khác nhau đến monolith hoặc services mới) rất phù hợp cho các hệ thống định hướng API (API-driven); **Event Interception** (service mới lắng nghe sự kiện từ monolith để dần thay thế logic) lý tưởng cho hệ thống định hướng sự kiện (event-driven); và **Asset-based** (tách tài nguyên tĩnh, thành phần giao diện trước) thích hợp cho các ứng dụng nặng về giao diện (frontend-heavy).

Với KBM — nơi đã có API Gateway (Spring Cloud Gateway, Ch.8) — **route-based Strangler Fig** là lựa chọn tự nhiên nhất cho LMS chính. Với DevOps Lab, routing lại dựa trên hostname/wildcard DNS ở mức khái quát. Hai kiểu gateway này cùng tồn tại, cho thấy việc chuyển đổi không nhất thiết dùng một pattern duy nhất cho mọi bounded context.

```
spring:
  cloud:
    gateway:
```

```

routes:
- id: new_exam_service_route
  uri: http://exam-service:8080
  order: 1
  predicates:
    - Path=/api/v2/exams/**
- id: legacy_monolith_fallback_route
  uri: http://kbm-monolith:8080
  order: 100
  predicates:
    - Path=/**

```

Listing 10.1: Cấu hình Spring Cloud Gateway áp dụng Strangler Fig (định tuyến **v2** sang service mới, còn lại về Monolith)

Chuyên đổi tăng dần

“Never do a big bang rewrite. Strangle the monolith incrementally — each step delivers value, each step is reversible, and the system is always running.”

— Sam Newman, *Monolith to Microservices* [4b, Ch.3]

10.2.4. KBM: Strangler Fig thực tế — từ **kbm-legacy-monolith** đến **kbm-***

Trạng thái ban đầu (Before) của KBM (đã giới thiệu ở §1.8) là mô hình *independent silo monoliths* — ba khối MVC rời rạc chạy cạnh nhau mà không chia sẻ dữ liệu hay danh tính, đúng mô hình mà Strangler Fig sinh ra để xử lý: phần quản lý đào tạo là ứng dụng **ASP.NET MVC** nguyên khối (**kbm-legacy-monolith**), còn các bộ chấm là những monolith MVC **tách rời** do nhóm tự viết — một code judge (C/C++ và Java, **không sandbox**) và một network judge (Java). Vì mỗi hệ là một khối riêng, chi phí bảo trì nhân lên theo số hệ thống chứ không được chia nhỏ.

Thay vì “Big Bang”, KBM bóc tách dần theo đúng mô hình Strangler Fig dựa trên định tuyến (route-based): API Gateway (Spring Cloud Gateway, Ch.8) định tuyến từng tính năng (capability) sang các dịch vụ **kbm-*** mới (**kbm-auth**, **kbm-academic**, **kbm-judge-sql**, **kbm-judge-network**, ...) trong khi phần còn lại của monolith vẫn phục vụ sinh viên. Mỗi bước tách tương ứng một mục trong danh mục (inventory) phân tích khoảng trống (gap analysis) được tổng hợp ở §10.5 — ranh giới bounded context làm rõ trước (Ch.2), rồi mới chuyển hướng định tuyến (route), đảm bảo mỗi bước nhỏ và có thể hoàn tác (rollback).

Một bước Strangler Fig điển hình là **kbm-judge-code**. Bộ chấm mã nguồn (code judge) **Gen 1** (di sản, tách từ **kbm-legacy-monolith**) do nhóm tự viết, chỉ hỗ trợ C/C++ và Java, và **chạy bài nộp trực tiếp trên môi trường OS của máy chủ — không có môi trường cách ly (sandbox)**: mã sinh viên có thể truy cập tài nguyên máy chủ, không giới hạn CPU/RAM. Đây vừa là lỗ hổng bảo mật (Ch.9) vừa là gánh nặng bảo trì.

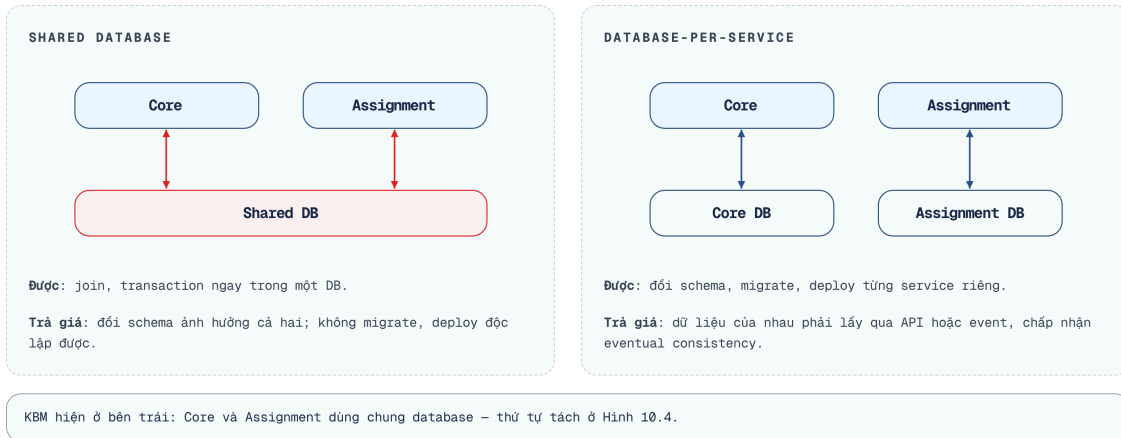
Thay vì tiếp tục chấp vá một bộ chấm tự xây, đội ngũ chọn **xây → mua/tích hợp (build → buy/adopt)**: **kbm-judge-code Gen 2** tích hợp **DOMjudge** — online judge mã nguồn mở chuẩn ICPC, đã được tích hợp sẵn môi trường cách ly (sandbox bằng cgroup/namespace) và đa ngôn ngữ. Quyết định này **loại bỏ hoàn toàn một lớp rủi ro**: cô lập sandbox và bảo trì compiler/runner được chuyển cho cộng đồng mã nguồn mở đã kiểm chứng, đội ngũ chỉ còn tích hợp và vận hành. Đây là minh họa rằng việc chuyển đổi không phải lúc nào cũng là “tách ra rồi tự viết lại” — đôi khi bước đúng nhất là **thay một thành phần tự xây bằng một sản phẩm đã được kiểm chứng**, loại bỏ hoàn toàn một gánh nặng thay vì chỉ di dời nó.

❗ Xây → Mua: thay bộ chăm code tự viết bằng DOMjudge

10.3. Tách Database — Thách thức Lớn nhất

10.3.1. Vấn đề: database coupling nguy hiểm hơn code coupling

Tách mã nguồn (code) thành các dịch vụ (services) không quá khó — tách API, triển khai (deploy) độc lập. Nhưng khi hai dịch vụ **chia sẻ cùng cơ sở dữ liệu** chúng vẫn bị ràng buộc (coupled) ở tầng sâu nhất:



Hình 10.3: Shared Database (coupled) vs Database-per-Service (decoupled)

Newman trong [4b, Ch.4] gọi shared database là **“the hardest part of decomposition”** — và LMS là ví dụ trực tiếp: Core Service và Assignment Service chia sẻ cùng database, những thay đổi về lược đồ (schema) ảnh hưởng đến cả hai, và hệ thống không thể triển khai các bản cập nhật cấu trúc cơ sở dữ liệu (database migration) một cách độc lập.

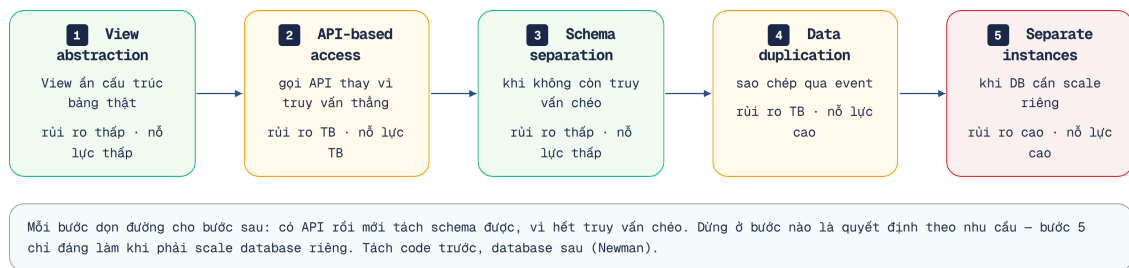
Từ Ch.7 (Database-per-Service), chúng ta đã biết nguyên tắc. Giờ nhìn từ góc độ chuyển đổi — **thủ tục thực hiện**.

#	Chiến lược (Strategy)	Mô tả	Rủi ro (Risk)	Nỗ lực (Effort)	Khi nào
1	View abstraction	Tạo View ẩn đi cấu trúc bảng thực tế	● Thấp	Thấp	Bước đầu tiên — khi service B cần dữ liệu của service A
2	API-based access	Service B gọi API service A thay vì truy vấn trực tiếp	● Trung bình	Trung bình	Khi thay thế việc gọi qua View/DB
3	Schema separation	Tách lược đồ (schema) trong cùng DB instance	● Thấp	Thấp	Khi không còn truy vấn chéo (nhờ API)

#	Chiến lược (Strategy)	Mô tả	Rủi ro (Risk)	Nỗ lực (Effort)	Khi nào
4	Data duplication	Sao chép dữ liệu thông qua các sự kiện (eventual consistency)	● Trung bình	Cao	Khi việc truy vấn chéo giữa các dịch vụ (cross-service queries) trở nên phức tạp
5	Separate instances	Database instance riêng cho mỗi service	● Cao	Cao	Bước cuối — khi cần mở rộng (scale) cơ sở dữ liệu riêng

Bảng 10.3: 5 chiến lược tách database — từ góc độ chuyển đổi

Sự dịch chuyển qua năm chiến lược trên được mô tả trong sơ đồ:



Hình 10.4: Thứ tự thực hiện 5 chiến lược tách database

Tiến trình năm bước trên giúp giảm thiểu rủi ro và cho phép đội ngũ phát triển làm quen dần với độ phức tạp của dữ liệu phân tán. Đây là thứ tự tổng quát theo hướng *API trước*: gỡ truy vấn chéo rồi mới tách schema. Ch.7 đặt nó cạnh hướng *schema trước* — dựng ranh giới rõ trong cùng DB server, tạm cho đọc chéo qua view rồi thay dần bằng API — và giải thích vì sao KBM, với hai service dùng chung một PostgreSQL và một nhóm nhỏ, chọn hướng sau (mục “Áp dụng cho KBM” ở Ch.7). Lộ trình Giai đoạn 4 ở cuối chương đi theo lựa chọn đó.

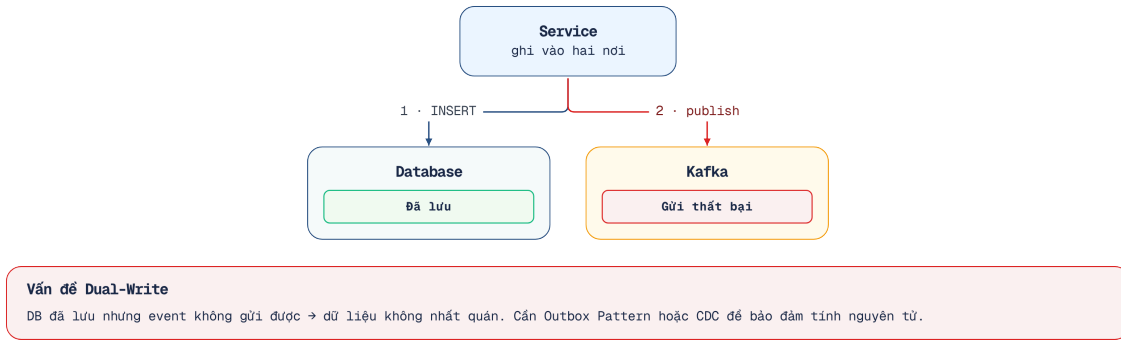
! Tách database tăng dần

“Don’t try to split the database and the code at the same time. Split the code first, then the database. Trying to do both simultaneously dramatically increases risk and complexity.”

— Sam Newman, *Monolith to Microservices* [4b, Ch.4]

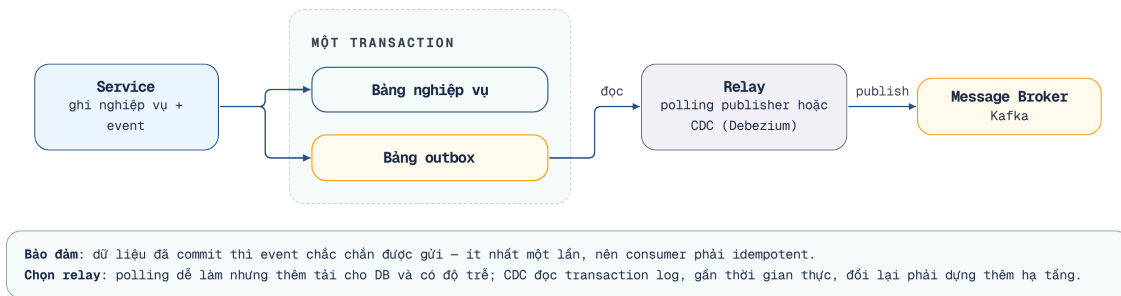
10.3.2. Cạm bẫy ghi kép (dual-write) và Outbox Pattern

Khi tách database, một anti-pattern phổ biến là **dual write**: dịch vụ vừa ghi vào cơ sở dữ liệu vừa phát tán sự kiện (publish event) — nếu một trong hai thất bại, dữ liệu sẽ không nhất quán.



Hình 10.5: Cạm bẫy ghi kép — DB thành công nhưng event thất bại

Trạng thái không nhất quán này đặc biệt nguy hiểm với việc nộp bài thi hoặc ghi nhận điểm số. Cách xử lý phổ biến là dùng chính database nội bộ làm điểm trung chuyển, tách thao tác ghi nghiệp vụ khỏi thao tác đẩy dữ liệu qua mạng. **Outbox Pattern** giải quyết vấn đề bằng cách ghi sự kiện vào bảng outbox cùng một giao dịch với dữ liệu nghiệp vụ chính; sau đó, một tiến trình ngầm sẽ đọc bảng này và đẩy thông điệp lên message broker:



Hình 10.6: Outbox Pattern — sự kiện được ghi cùng giao dịch với dữ liệu nghiệp vụ

Có hai cách để tiến trình ngầm đọc và đẩy sự kiện từ bảng outbox. Phương pháp thứ nhất là **Polling publisher**, sử dụng một luồng chạy ngầm để liên tục kiểm tra bảng outbox; cách này dễ triển khai nhưng tạo thêm tải cho cơ sở dữ liệu và có độ trễ nhất định. Phương pháp thứ hai tiên tiến hơn là sử dụng **CDC (Change Data Capture)** như Debezium để đọc trực tiếp từ transaction log; cách này đảm bảo tính thời gian thực và loại bỏ hao tổn hiệu năng từ việc polling, nhưng đòi hỏi phải thiết lập hạ tầng phức tạp hơn.

💡 Outbox Pattern

Giống như sinh viên nộp bài thi (business data) và ký tên vào danh sách điểm danh (event) CÙNG MỘT LÚC tại bàn giáo viên. Sau đó, một giám thị khác (tiến trình ngầm) mới cầm danh sách điểm danh đó báo cáo cho Phòng Đào tạo (Message Broker).

Để bài làm của sinh viên không bị mất khi sự cố mạng xảy ra giữa lúc ghi dữ liệu và phát sự kiện, dưới đây là cách cài đặt Outbox Pattern vào hệ thống nộp bài:

```
@Service
public class SubmissionService {
    private final SubmissionRepository submissionRepo;
    private final OutboxRepository outboxRepo;

    @Transactional
```

```

public void submitExam(Submission submission) {
    // 1. Lưu business data
    submissionRepo.save(submission);

    // 2. Lưu event vào bảng Outbox (cùng transaction)
    OutboxEvent event = new OutboxEvent(
        "SubmissionCreated",
        toJson(submission)
    );
    outboxRepo.save(event);
}
}

```

Listing 10.2: Mã nguồn Transactional Outbox Pattern (lưu cả hai thực thể trong một giao dịch)

KBM chưa có Outbox Pattern

KBM hiện dùng **dual write**: Core Service lưu bài nộp vào DB rồi gọi `submitProducer.send()` đến Kafka. Nếu Kafka không khả dụng (unavailable) tại thời điểm đó, bài nộp (submission) đã lưu nhưng Judge Service không nhận được bài — sinh viên thấy “đã nộp” nhưng không nhận kết quả chấm.

Lộ trình chuyển đổi (Migration path) (1) Ngắn hạn — cơ chế thử lại (retry logic) cho Kafka producer (đã có nhưng cần được kiểm chứng lại một cách kỹ lưỡng), (2) Trung hạn — bảng outbox + polling publisher, (3) Dài hạn — Debezium CDC. Với lưu lượng truy cập (traffic) thấp của KBM hiện tại, polling publisher (trung hạn) đủ tốt — Debezium chỉ cần khi mở rộng quy mô (scale) trên môi trường thực tế lớn.

10.3.3. CDC cho chuyển đổi dữ liệu

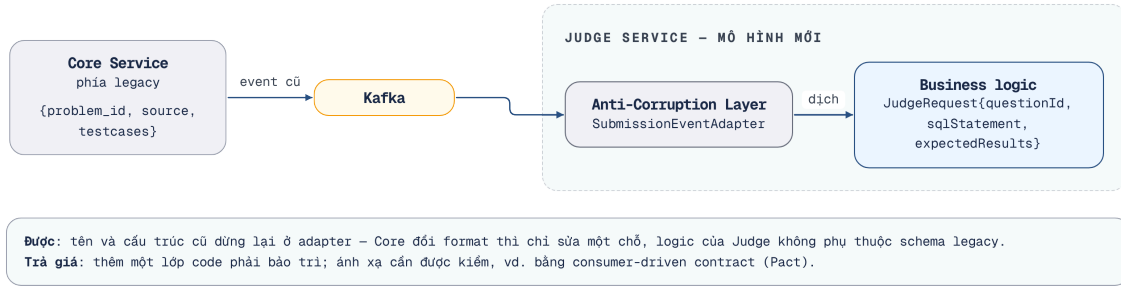
Lưu ý quan trọng: Trong chương này, chúng ta mới nhắc CDC trong ngữ cảnh Outbox Pattern. Tuy nhiên, CDC (như Debezium) thực chất là công cụ lõi để **đồng bộ dữ liệu thời gian thực (Real-time Sync)** khi phân tách cơ sở dữ liệu. Monolith vẫn tiếp tục ghi vào DB cũ, CDC sẽ nắm bắt (capture) các thay đổi đó và đẩy sang DB của Microservice mới liên tục. Nhờ vậy, cửa sổ gián đoạn khi chuyển đổi (cut-over) được rút xuống rất nhỏ (near-zero downtime): chỉ cần chặn thao tác ghi vài giây để CDC đuổi kịp độ trễ sao chép (replication) rồi mới chuyển hướng. Zero downtime tuyệt đối đòi hỏi thêm dual-write và cơ chế xử lý xung đột — chi phí thường không tương xứng ở quy mô KBM.

10.4. Anti-Corruption Layer và các pattern chuyển đổi

Khi dịch vụ mới bắt buộc phải gọi đến hệ thống di sản (legacy), chúng ta cần một lớp đệm trung gian để chuyển đổi mô hình dữ liệu, ngăn chặn thiết kế lạc hậu của hệ thống cũ ảnh hưởng đến kiến trúc mới.

10.4.1. Anti-Corruption Layer (ACL) — Bảo vệ ranh giới

Khi tách dịch vụ mới ra khỏi monolith, dịch vụ mới cần giao tiếp với mã nguồn di sản (legacy) — nhưng **không nên để mô hình (model) cũ ảnh hưởng tiêu cực đến mã nguồn mới**. Eric Evans trong [6] định nghĩa **Anti-Corruption Layer (ACL)**: một lớp dịch ngôn ngữ giữa hai bounded contexts có mô hình (model) khác nhau.



Hình 10.7: Anti-Corruption Layer — lớp dịch giữa service mới và legacy

Ví dụ KBM: Judge Service nhận các bài nộp (submissions) từ Core Service qua Kafka. Core Service truyền dữ liệu theo định dạng (format) `{problem_id, source, testcases}` (cách đặt tên cũ - legacy naming). Judge Service có thể: (1) × dùng trực tiếp tên trường dữ liệu di sản (legacy field names) trong mã nguồn — bị ràng buộc (coupling) với các quyết định của hệ thống di sản, hoặc (2) ✓ tạo adapter dịch sang mô hình nội bộ `JudgeRequest{questionId, sqlStatement, expectedResults}` — ranh giới rõ ràng (clean boundary).

ACL hoạt động như một “Thông dịch viên” hoặc hệ thống quy đổi tín chỉ: nó giúp dịch vụ mới tích hợp với hệ thống cũ (Monolith) mà không bị xung đột dữ liệu vì khác biệt mô hình miền (Domain Model).

Tương tự như việc Phòng Khảo thí tự động quy đổi điểm cho sinh viên mà không bắt họ phải nộp lại hồ sơ, đoạn mã dưới đây minh họa cách một Adapter thực hiện chuyển đổi luồng dữ liệu một cách trong suốt:

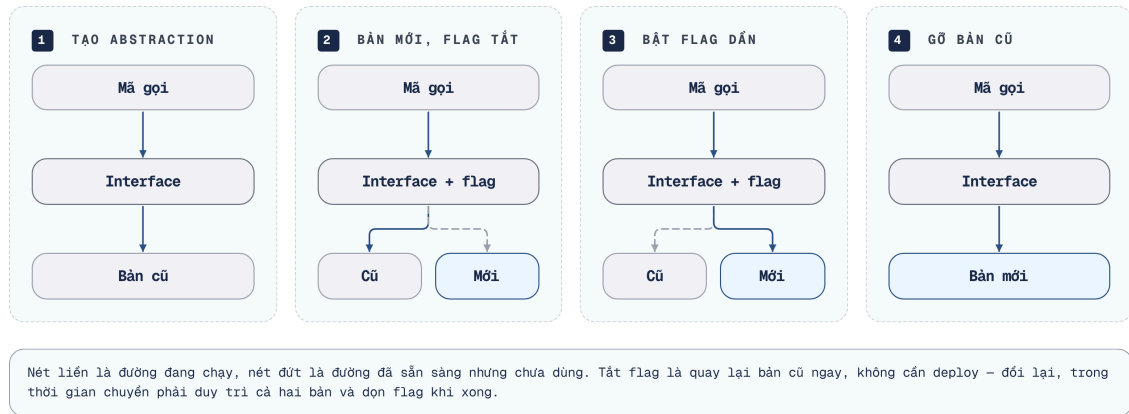
```
@Component
public class SubmissionEventAdapter {
    // Dịch từ model cũ (Monolith) sang model mới (Judge Service)
    public JudgeRequest translate(LegacySubmissionEvent legacyEvent) {
        return new JudgeRequest(
            legacyEvent.getProblemId(),
            legacyEvent.getSource(),
            parseTestcases(legacyEvent.getTestcases()) // Phân tích định dạng cũ
        );
    }
}
```

Listing 10.3: Adapter trong Anti-Corruption Layer dịch sự kiện cũ (legacy event) sang mô hình mới

Khi bóc tách service, để đảm bảo việc thay đổi ở service mới không vô tình phá vỡ monolith cũ đang gọi nó, chúng ta nên áp dụng **Consumer-Driven Contracts** (ví dụ framework Pact) để tự động hóa việc kiểm tra tính tương thích API giữa hai hệ thống.

10.4.2. Branch by Abstraction

Newman trong [4b, Ch.3] đề xuất **Branch by Abstraction** cho việc chuyển đổi an toàn:



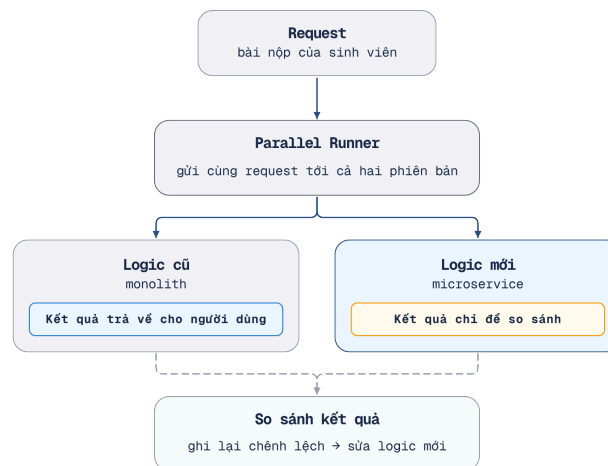
Hình 10.8: Branch by Abstraction — chuyển đổi an toàn qua feature flag

1. **Tạo lớp trừu tượng** (abstraction layer, interface) che chắn cho phần hiện thực cũ
2. **Hiện thực hóa phiên bản mới** phía sau cùng interface, kiểm soát bằng feature flag
3. **Chuyển lưu lượng** dần sang phiên bản mới và kiểm chứng
4. **Gỡ bỏ** phiên bản cũ khi đã chắc chắn về tính ổn định

Để quản lý Feature Flag chuyên nghiệp trong Java, thay vì dùng `if-else` cứng và cấu hình properties, chúng ta nên dùng các thư viện chuyên dụng như **Togglz**, **Unleash**, hoặc **Spring Cloud Config**. Các thư viện này cho phép bật/tắt luồng logic ngay tại runtime mà không cần khởi động lại dịch vụ.

10.4.3. Parallel Run — Kiểm chứng trước khi chuyển đổi

Parallel Run — chiến lược cho phép **chạy đồng thời** cả phiên bản hiện thực cũ và mới, sau đó đối chiếu kết quả đầu ra. Ở cấp độ hạ tầng, API Gateway có thể áp dụng **Traffic Shadowing (Dark Launching)**: nhân bản âm thầm (shadow) một luồng lưu lượng (traffic) thực tế và chuyển hướng đến dịch vụ mới để kiểm tra tải và kiểm tra logic mà người dùng không hề hay biết.



Hình 10.9: Parallel Run — chạy song song logic cũ và mới, so sánh kết quả

- Phiên bản cũ **trả kết quả (response)** cho người dùng (đảm bảo an toàn - production-safe)
- Phiên bản mới chạy **song song** — kết quả trả về bị loại bỏ (discard)
- So sánh hai kết quả — ghi nhận (log) các điểm sai lệch (mismatches)
- Khi tỷ lệ sai lệch (mismatch rate) $\approx 0\%$ → chuyển đổi (switch) sang phiên bản mới

⚠ Parallel Run chỉ an toàn với thao tác đọc

Với thao tác ghi (INSERT, UPDATE), chạy song song sẽ tạo tác dụng phụ kép (ghi hai lần). Để kiểm chứng an toàn, hãy dùng kỹ thuật **Shadow Database** hoặc **Discard Writes** — loại bỏ mọi thao tác ghi của phiên bản mới.

Áp dụng cho KBM: nếu cần viết lại `CompareUtil` (logic so sánh kết quả SQL — quan trọng nhất (critical) trong hệ thống), Parallel Run cho phép chạy phiên bản `CompareUtil` cũ và mới song song trên mỗi bài nộp, so sánh kết quả để đảm bảo logic chấm điểm mới hoàn toàn khớp trước khi chính thức chuyển đổi. Tuy nhiên, hệ thống **BẮT BUỘC** phải hủy bỏ (discard) các thao tác ghi (write/side-effects) của phiên bản mới để tránh làm hỏng dữ liệu thật.

Chiến lược Parallel Run giúp xác minh tính chính xác của phiên bản mới bằng dữ liệu thực tế (production data) trước khi chuyển hướng hoàn toàn lưu lượng truy cập (cut-over).

💡 Chuyên đổi phải hoàn tác được

“Every migration step should be reversible. If you can’t roll back a change easily, you’re taking on too much risk at once. Feature flags, parallel runs, and incremental routing all serve the same purpose: *making it safe to fail*”

— Sam Newman, *Monolith to Microservices* [4b, Ch.3]

10.5. Quá trình chuyên đổi kiến trúc tại KBM

Phần này tổng hợp toàn bộ phân tích khoảng trống từ Chương 1–9 thành một danh mục duy nhất, rồi sắp xếp thành lộ trình chuyển đổi bốn giai đoạn ưu tiên theo Impact × Effort — công cụ thực thi cho đội ngũ KBM.

10.5.1. Tổng hợp các phân tích khoảng trống

Xuyên suốt các chương trước, chúng ta đã phân tích khoảng trống (gap) giữa KBM hiện tại và các giải pháp khuyến nghị. Bảng dưới tổng hợp toàn bộ — **đây là danh mục cho lộ trình chuyên đổi**.

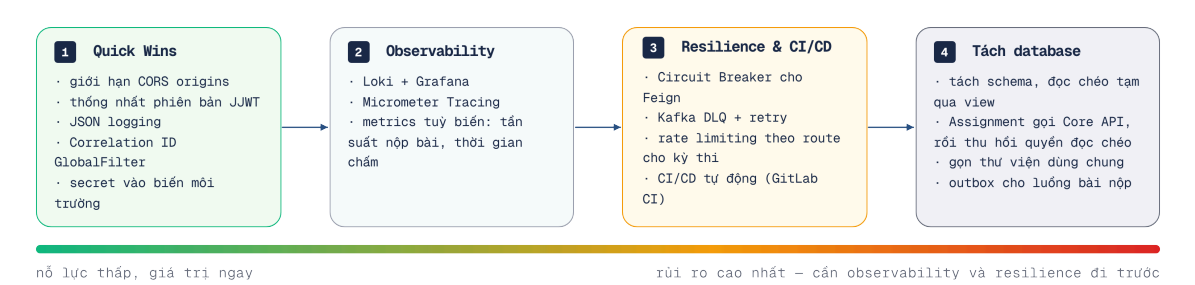
Chương	Khoảng trống	Mức độ	Tham chiếu
Ch.2	7 contexts rõ hơn nhưng thư viện dùng chung vẫn coupling	! Trung bình	\$2.5
Ch.3	Đặt tên/phiên bản API chưa nhất quán; giao ước worker của SQL Judge cần chuẩn hóa	! Trung bình	\$3.9
Ch.4	SQL Judge coordinator/worker và tích hợp bên ngoài cần ACL/resilience rõ hơn	! Trung bình	\$4.5
Ch.5	Kafka pipeline cần DLQ/idempotency; SQL Judge dispatch và DevOps Lab queue cần chọn broker theo khối lượng công việc (workload)	! Trung bình	\$5.8
Ch.6	Luồng Choreography saga chưa có cơ chế giám sát hiệu quả	● Thấp	\$6.6

Chương	Khoảng trống	Mức độ	Tham chiếu
Ch.7	Shared database (Core ↔ Assignment)	Cao	§7.6
Ch.8	Spring Cloud Gateway + Go router cần thống nhất chính sách ở biên	Cao	§8.5
Ch.9	HS256 shared secret, token storage, secrets, SSO đa nền tảng	Cao	§9.6
Ch.11	Observability có Actuator/Prometheus một phần, chưa có truy vết đầu cuối/tập trung log	Cao	§11.9
Ch.12	LMS chính Docker Compose; DevOps Lab k3s/Sysbox; CI/CD và rollback chưa đồng đều	Cao	§12.7

Bảng 10.4: Tổng hợp các phân tích khoảng trống xuyên suốt Ch.1–12

10.5.2. Lộ trình chuyển đổi – 4 giai đoạn

Nguyên tắc ưu tiên: **Quick Wins trước, các thay đổi rủi ro cao (High-Risk) thực hiện sau.** Mỗi giai đoạn mang lại giá trị ngay lập tức – không cần đợi hoàn thành toàn bộ.



Hình 10.10: Lộ trình chuyển đổi 4 giai đoạn – Quick Wins → Observability → Resilience → Decomposition

Việc chia nhỏ thành bốn giai đoạn giúp đội ngũ ưu tiên giải quyết các vấn đề có tác động cao nhưng đòi hỏi nỗ lực thấp (High Impact / Low Effort) trước, và hoãn lại các thay đổi cơ sở dữ liệu rủi ro cao cho đến khi có đủ cơ chế giám sát.

10.5.2.1. Giai đoạn 1 – Các cải tiến nhanh (Quick Wins): nỗ lực thấp, tác động cao

Giai đoạn mở đầu tập trung vào những thay đổi nhỏ, tốn ít công sức nhưng mang lại hiệu quả bảo mật và vận hành đáng kể, không đòi hỏi thay đổi kiến trúc cốt lõi.

#	Hành động	Khoảng trống	Nỗ lực	Tác động
1	Giới hạn nguồn gốc CORS (CORS origins)	Ch.8	30 phút	Bịt lỗ hổng bảo mật
2	Thống nhất phiên bản JWT trong parent POM	Ch.8	1 giờ	Ngăn lỗi sai lệch phiên bản
3	Chuyển sang JSON logging (Logback encoder)	Ch.11	2 giờ	Log có thể tìm kiếm

#	Hành động	Khoảng trống	Nỗ lực	Tác động
4	Thêm Correlation ID GlobalFilter	Ch.8, Ch.11	4 giờ	Gỡ lỗi chéo dịch vụ
5	Đưa thông tin nhạy cảm vào biến môi trường	Ch.9	2 giờ	Không bị lộ (leak) qua Git

Bảng 10.5: Giai đoạn 1 — Quick Wins

Triết lý Giai đoạn 1: Không thay đổi kiến trúc tổng thể, không can thiệp sâu vào logic cốt lõi — chỉ tập trung **chuẩn hóa cấu hình và tăng cường bảo mật**. Một kỹ sư duy nhất cũng đủ sức hoàn thành nhóm công việc này trong một đến hai sprint — và đây là bước đệm cần thiết trước khi bắt tay vào các phần phân tách phức tạp hơn.

10.5.2.2. Giai đoạn 2 — Nền tảng Observability

Sau Giai đoạn 1, hệ thống cần được trang bị khả năng giám sát đầy đủ trước khi tiến hành phân tách các thành phần phức tạp.

#	Hành động	Khoảng trống	Nỗ lực	Tác động
1	Triển khai Loki + Grafana	Ch.11	1-2 ngày	Tra cứu log tập trung
2	Thêm Micrometer Tracing	Ch.11	1 ngày	Biết nút thắt cổ chai (bottleneck) ở đâu
3	Metrics tùy biến (tần suất nộp bài, thời gian chấm)	Ch.11	2 ngày	Giám sát chủ động (Proactive monitoring)

Bảng 10.6: Giai đoạn 2 — Nền tảng Observability

Triết lý Giai đoạn 2: Trước khi thực hiện các thay đổi kiến trúc phức tạp ở Giai đoạn 3 và 4, hệ thống cần có cơ chế giám sát tốt. Dashboard và hệ thống log tập trung là công cụ quan trọng để gỡ lỗi.

10.5.2.3. Giai đoạn 3 — Resilience & CI/CD

Với khả năng giám sát đã sẵn sàng, bước tiếp theo là xây dựng cơ chế phòng thủ để ứng dụng chịu được lỗi và tự động hóa quy trình phân phối.

#	Hành động	Khoảng trống	Nỗ lực	Tác động
1	Resilience4j Circuit Breaker cho Feign clients	Ch.4	2-3 ngày	Ngăn chặn lỗi dây chuyền (cascading failures)
2	Kafka Dead Letter Queue + retry	Ch.5	2 ngày	Không mất thông điệp (messages)
3	Tinh chỉnh chính sách rate limiting cho kỳ thi (theo route)	Ch.8	1 ngày	Bảo vệ tốt hơn khi lạm dụng (abuse)
4	Hoàn thiện CI/CD deployment tự động (GitLab CI)	Ch.12	3-5 ngày	Tự động hóa biên dịch và triển khai

Bảng 10.7: Giai đoạn 3 — Resilience & CI/CD

Triết lý Giai đoạn 3: Hệ thống phải chứng minh được **tính bền bỉ trước khi bị xé nhỏ**. Việc phân tách cơ sở dữ liệu khi chưa có cơ chế ngắt mạch (circuit breaker) hay hệ thống cảnh báo sẽ dễ gây ra lỗi dây chuyền một khi có bất kỳ dịch vụ nào gặp sự cố nghẽn mạng.

10.5.2.4. Giai đoạn 4 — Phân tách cơ sở dữ liệu (Database Decomposition): thận trọng nhất

Bước cuối cùng — tháo gỡ sự phụ thuộc ở tầng dữ liệu — là thử thách lớn nhất, và đòi hỏi sự ổn định từ ba giai đoạn trước.

#	Hành động	Khoảng trống	Nỗ lực	Tác động
1	Schema separation (<code>core_schema</code> , <code>assignment_schema</code>); truy vấn chéo còn lại tạm đọc qua view	Ch.7	1-2 tuần	Ranh giới thấy được, đếm được truy vấn chéo
2	API-based access (Assignment gọi Core API); hết truy vấn chéo thì thu hồi quyền đọc chéo	Ch.7	2-3 tuần	Loại bỏ truy cập DB trực tiếp
3	Refactor thư viện dùng chung (chỉ giữ DTO, ngoại lệ)	Ch.2	2-3 tuần	Giảm thiểu sự phụ thuộc (coupling)
4	Outbox pattern cho luồng xử lý bài nộp	Ch.5, Ch.6	1-2 tuần	Giao tiếp tin cậy (Reliable messaging)

Bảng 10.8: Giai đoạn 4 — Phân tách cơ sở dữ liệu

Triết lý Giai đoạn 4: Đây là tập hợp những thay đổi chứa đựng **nhiều rủi ro nhất** trong toàn bộ lộ trình. Chúng ta dựa vào nền tảng quan sát (Giai đoạn 2) và tính bền bỉ (Giai đoạn 3) để thực hiện bước này. Mỗi thay đổi cần được triển khai từ từ, theo dõi qua dashboard, khi ổn định mới tiến hành bước tiếp theo.

10.5.3. Ma trận quyết định — Ưu tiên theo tác động × nỗ lực



Hình 10.11: Ma trận quyết định — ưu tiên theo tác động × nỗ lực

Ma trận trên giúp vạch ra chiến lược nâng cấp LMS, cân đối giữa giới hạn nhân sự IT và lợi ích thiết thực mang lại cho sinh viên.

💡 **Chuyển đổi là Marathon**

Đừng cố gắng khắc phục mọi khiếm khuyết (gap) cùng lúc. Trong mỗi chu kỳ phát triển (sprint), chỉ nên ưu tiên 1-2 hạng mục thuộc nhóm “Tác động cao, Nỗ lực thấp (High Impact, Low Effort)” trước. Khi đã hết các thành quả nhanh (quick wins), mới chuyển sang nhóm “Tác động cao, Nỗ lực cao” — và luôn phải đảm bảo khả năng quan sát (observability) trước khi thực hiện các thay đổi lớn.

10.6. Sai lầm thường gặp khi chuyển đổi

Ngoài 4 anti-patterns ở 10.2 (§10.1), quá trình chuyển đổi có những rủi ro cần được nhận diện và xử lý sớm. Dưới đây là những sai lầm phổ biến nhất trong quá trình chuyển đổi kiến trúc:

Distributed Monolith – Tách các dịch vụ (services) nhưng vẫn dùng chung cơ sở dữ liệu (shared database), dẫn đến quy trình triển khai phải đồng bộ. Newman trong [4b, Ch.1] gọi đây là “the worst of both worlds”. *Phòng tránh* bằng cách xem nguyên tắc database-per-service là tiêu chí quyết định.

Big Bang Rewrite – Viết lại toàn bộ từ đầu (from scratch), ngày chuyển đổi (“switch”) không bao giờ đến. *Phòng tránh* bằng cách sử dụng Strangler Fig (§10.2) — mỗi chu kỳ phát triển (sprint) tách một phần, hệ thống luôn ở trạng thái sẵn sàng triển khai (deployable).

Bỏ qua Conway’s Law – Phân tách thành các dịch vụ độc lập nhưng cấu trúc đội ngũ phát triển vẫn như cũ. *Phòng tránh* bằng cách tổ chức lại đội ngũ theo các bounded context (Ch.2) — mỗi nhóm chỉ nên sở hữu 1-2 dịch vụ.

“Lift and Shift” không thiết kế lại – Sao chép nguyên bản mã nguồn vào các container, và ngộ nhận đó là kiến trúc microservices. *Phòng tránh* bằng cách sử dụng Anti-Corruption Layer (§10.4) ngăn chặn mô hình dữ liệu cũ (legacy model) xâm nhập vào kiến trúc mới.

Hạ tầng phức tạp quá mức cần thiết – Triển khai K8s + Istio cho 3 dịch vụ với 100 yêu cầu/phút. *Phòng tránh* bằng cách bắt đầu đơn giản (Docker Compose, Ch.12), chỉ mở rộng quy mô (scale) khi lưu lượng truy cập thực sự đòi hỏi.

💡 **Interactive Demo**

Xem minh họa động tại companion web:

Chiến lược Strangler Fig Pattern – — strangler-fig-migration.html

Transactional Outbox – — outbox-pattern.html

CDC và Outbox – — cdc-outbox.html

10.7. Tổng kết

Chuyển đổi từ monolith sang microservices không phải quyết định kỹ thuật thuần túy — đây là sự kết hợp giữa **tổ chức đội ngũ (team organization)** (Conway’s Law, Ch.2), **hiểu biết về miền nghiệp vụ (domain understanding)** (Bounded Contexts, Ch.2), **các mô hình giao tiếp (communication patterns)** (Ch.3-6), **chiến lược dữ liệu (data strategy)** (Ch.7), **sự sẵn sàng của hạ tầng (infrastructure readiness)** (Ch.8-9), và **mức độ trưởng thành trong vận hành (operational maturity)** (Ch.11-12).

Bài học quan trọng nhất: “**Monolith First**” — bắt đầu đơn giản, chỉ chuyển đổi (migrate) khi có lý do cụ thể, thực hiện chuyển đổi tăng dần (Strangler Fig), và mỗi bước đều phải có khả năng hoàn tác (reversible). Việc đập đi xây lại toàn bộ (Big Bang rewrites) gần như luôn thất bại; Strangler Fig gần như luôn thành

công — vì nó cho phép chấp nhận các rủi ro ở quy mô nhỏ, rút kinh nghiệm nhanh chóng và liên tục mang lại giá trị thực tiễn.

Tách database là thách thức lớn nhất — sự phụ thuộc về mã nguồn có thể giải quyết qua tái cấu trúc (refactoring), nhưng sự phụ thuộc về dữ liệu đòi hỏi một chiến lược chuyển đổi vô cùng cẩn trọng: view abstraction → API-based access → schema separation → data duplication → separate instances. Outbox Pattern giải quyết bài toán giao tiếp tin cậy (reliable messaging) trong quá trình chuyển đổi — giúp tránh cái bẫy ghi dữ liệu kép (dual-write pitfall).

Anti-Corruption Layer, Branch by Abstraction, và Parallel Run là bộ công cụ cho việc chuyển đổi an toàn — mỗi pattern phục vụ mục đích khác nhau nhưng chia sẻ nguyên tắc chung: **làm cho quá trình chuyển đổi có thể đảo ngược, chia nhỏ từng bước tiến hành, và đảm bảo hệ thống luôn trong trạng thái sẵn sàng triển khai.**

Lộ trình chuyển đổi cho KBM — tổng hợp từ các phân tích khoảng trống (gap analyses) từ Ch.1-9 — cho thấy: các Quick Wins như CORS, JWT, log có cấu trúc (structured logging) có thể hoàn thành trong 1-2 tuần, mang lại giá trị ngay. Phân tách cơ sở dữ liệu (Database decomposition) — thay đổi lớn nhất — cần được thiết kế dựa trên nền tảng về khả năng quan sát (observability) và tính bền bỉ (resilience). DevOps Lab cho thấy cùng một hệ thống có thể cần chiến lược khác nhau theo bounded context: Docker Compose đủ tốt cho LMS chính, trong khi việc cô lập phòng thí nghiệm cần k3s/Sysbox. Mỗi quyết định thiết kế đều là một sự đánh đổi — không có phương án “đúng tuyệt đối”, chỉ có phương án “phù hợp nhất với bối cảnh hệ thống tại thời điểm đó”.

Ở Chương 11, chúng ta sẽ xem cách **quan sát và giám sát** hệ thống sau khi chuyển đổi — logging, metrics, tracing — ba kỹ thuật trọng tâm giúp phát hiện và xử lý vấn đề trước khi người dùng bị ảnh hưởng.

10.8. Đọc thêm

Sách tham khảo chính:

1. [4b] Sam Newman, *Monolith to Microservices* — Toàn bộ sách: decomposition patterns, database splitting, migration strategies
2. [2a] Chris Richardson, *Microservices Patterns* 1st Ed. — Ch.13: Refactoring to Microservices — Strangler Fig, Anti-Corruption Layer
3. [4a] Sam Newman, *Building Microservices* — Ch.2: How to Model Microservices (decomposition), Ch.3: Splitting the Monolith

Sách bổ trợ:

4. [6] Eric Evans, *Domain-Driven Design* — Ch.14: Anti-Corruption Layer, Context Mapping
5. [3] Ronnie Mitra & Irakli Nadareishvili, *Microservices: Up and Running* — Ch.3: Architecture Design, migration planning
6. [2b] Chris Richardson, *Microservices Patterns* 2nd Ed. — Ch.2: Migration Anti-patterns; Ch.13: Refactoring to Microservices; Ch.21: Strangler Fig (updated)

Nguồn trực tuyến:

- [W2] Martin Fowler, “MonolithFirst” — martinfowler.com/bliki/MonolithFirst.html
- [W8] Martin Fowler, “StranglerFigApplication” — martinfowler.com/bliki/StranglerFigApplication.html
- Sam Newman, “Breaking Apart the Monolith” — samnewman.io
- Debezium (CDC) — debezium.io
- Martin Fowler, “BranchByAbstraction” — martinfowler.com/bliki/BranchByAbstraction.html

Bài tập: Xem Phụ lục Bài tập để luyện tập:

10-0 – Ôn tập: Các chiến lược chuyển đổi ([L1])

10-1 – Case Study: Amazon — Monolith → SOA → Microservices (2002–2020) ([L3])

10-2 – Outbox Pattern vs Event Sourcing vs Change Data Capture (Trade-off Analysis [L2])

10-3 – Đồng bộ dữ liệu và Strangler Fig (Design [L3])

Tài liệu tham khảo

- Richardson, C. (2018). *Microservices Patterns: With examples in Java*, 1st Edition. Manning Publications.
- Richardson, C. (2025). *Microservices Patterns: With examples in Java*, 2nd Edition (bản MEAP đang cập nhật; nội dung trích dẫn truy cập 07/2026). Manning Publications.
- Newman, S. (2019). *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media.